

Команды CREATE и ALTER

ОСНОВЫ SQL

Реляционные базы данных

Skillbox

Оглавление

Команда CREATE	3
Команда ALTER	13
Связь между таблицами через внешние ключи	16

Команда CREATE

Работа с базой данных начинается с её создания. Для этого используется команда [CREATE DATABASE](#). Это первый шаг в организации хранилища данных, позволяющий структурировать данные и управлять ими.

```
CREATE DATABASE name [OPTIONS]
```

- **CREATE DATABASE** — команда для создания новой базы данных.
- **name** — имя новой базы данных, которое должно быть уникальным в рамках сервера базы данных.
- **[OPTIONS]** — дополнительные параметры для настройки базы данных (необязательно).

Следующий шаг после создания базы данных — создание таблиц для хранения данных. Для этого используется команда [CREATE TABLE](#). Она позволяет создать новую таблицу в базе данных и определить её структуру, задавая названия столбцов и их типы данных.

Почему это важно? Тип данных определяет, какие значения могут храниться в колонках. Например, столбец с типом **INT** будет хранить только целые числа, а столбец с типом **VARCHAR(255)** — текст длиной до 255 символов.

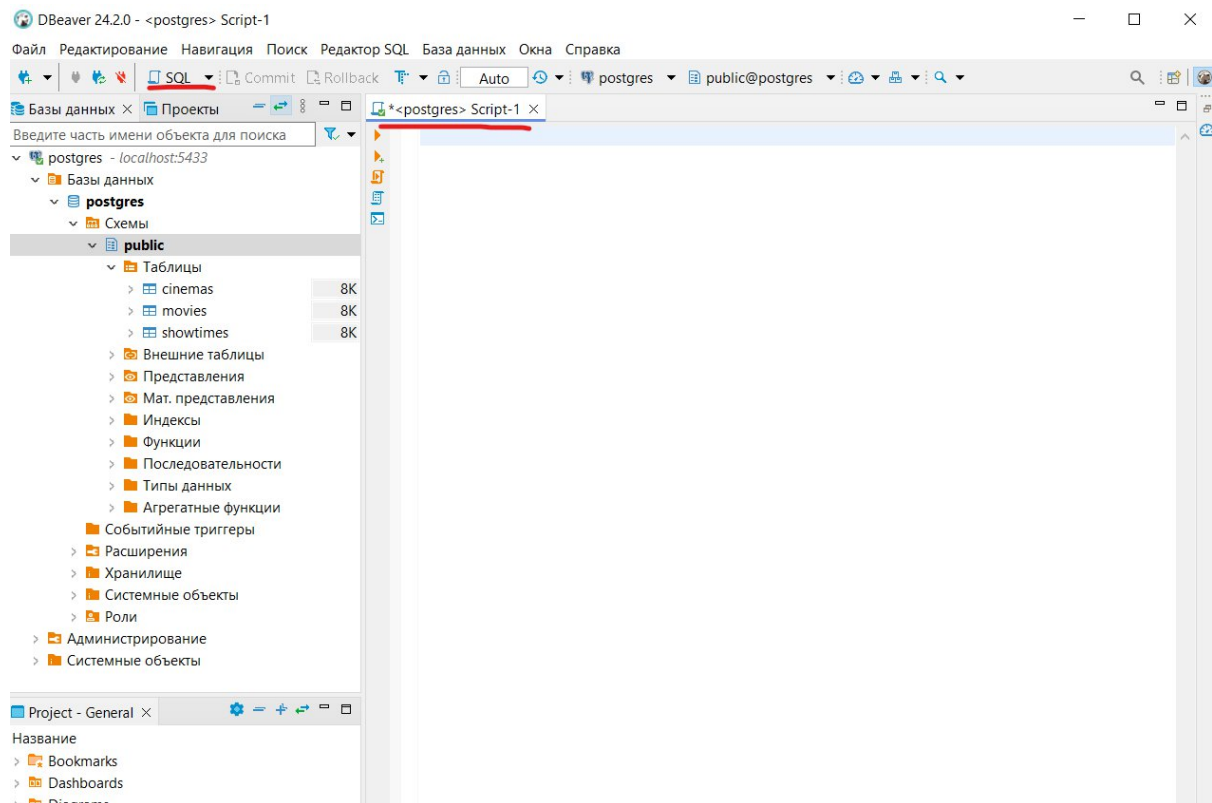
Синтаксис команды:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
);
```

- **CREATE TABLE** — создание новой таблицы.
- **table_name** — имя новой таблицы.
- **column1, column2 ...** — названия столбцов таблицы.
- **datatype** — тип данных в столбце.
- **constraint** — ограничения для столбца (необязательно).

Представьте, что мы создаём базу данных для кинотеатра. Для начала спроектируем таблицу, которая будет хранить основную информацию о фильмах. Ранее мы использовали графический интерфейс для создания таблиц, но теперь пришло время сделать это с помощью SQL-скрипта, который позволяет более гибко управлять данными и изменять структуру по мере необходимости. Выполним следующие шаги:

1. Открываем SQL-редактор в DBeaver. Открываем DBeaver и переходим в раздел SQL Editor, где можно вводить и выполнять SQL-запросы.

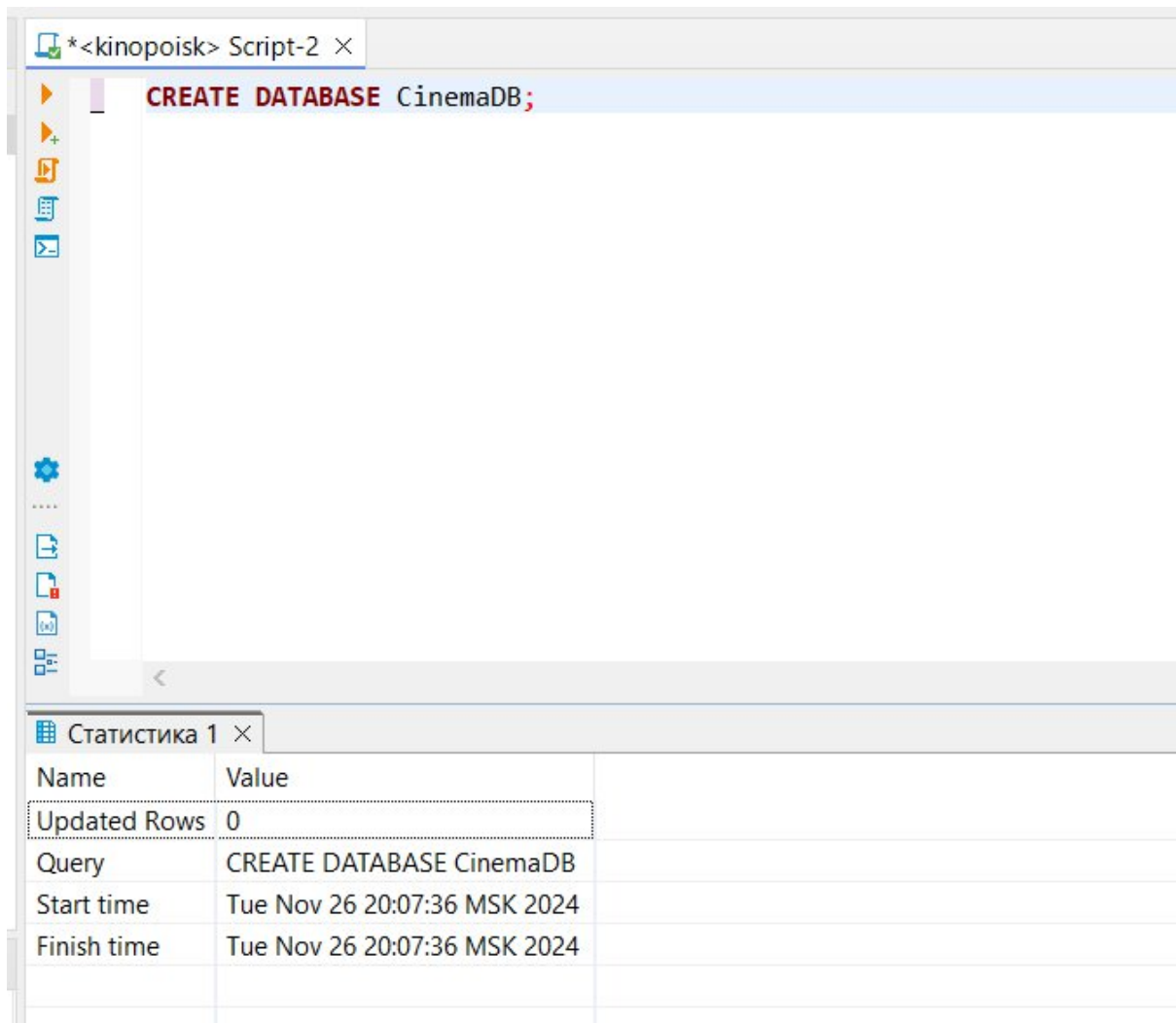


2. Создаём базу данных. Для начала создадим новую базу данных, в которой будут храниться таблицы с информацией о фильмах и актёрах. Выполним следующий скрипт:

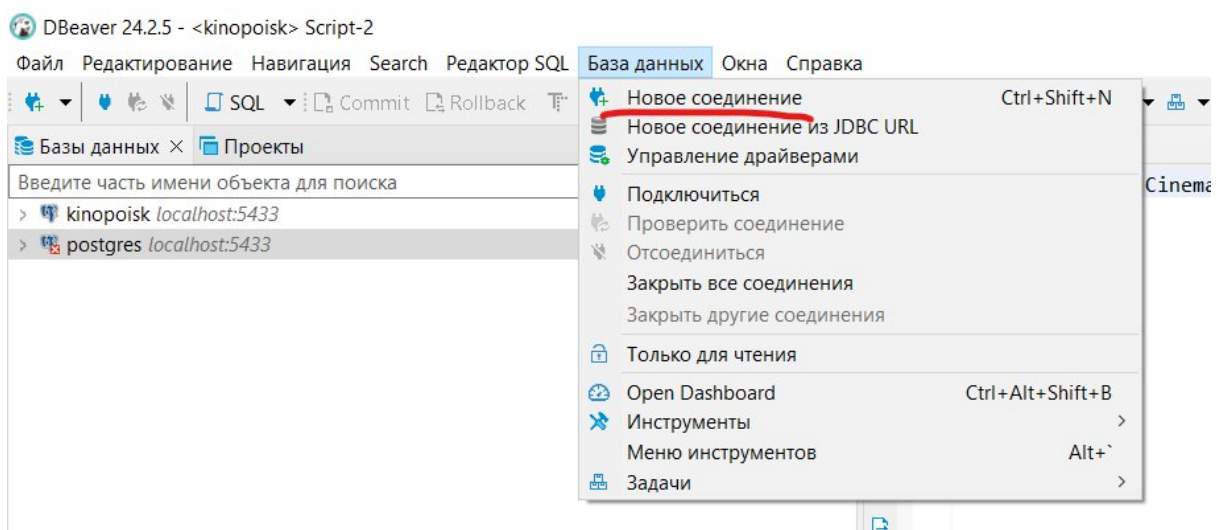
```
CREATE DATABASE CinemaDB WITH ENCODING 'UTF8';
```

Команда создаёт новую базу данных с названием **CinemaDB**. Эта база данных будет хранить данные в кодировке UTF-8, что обеспечивает поддержку символов разных языков, включая русский и английский. Кодировка UTF-8 особенно важна, если предполагается работа с текстовыми данными, содержащими символы различных алфавитов.

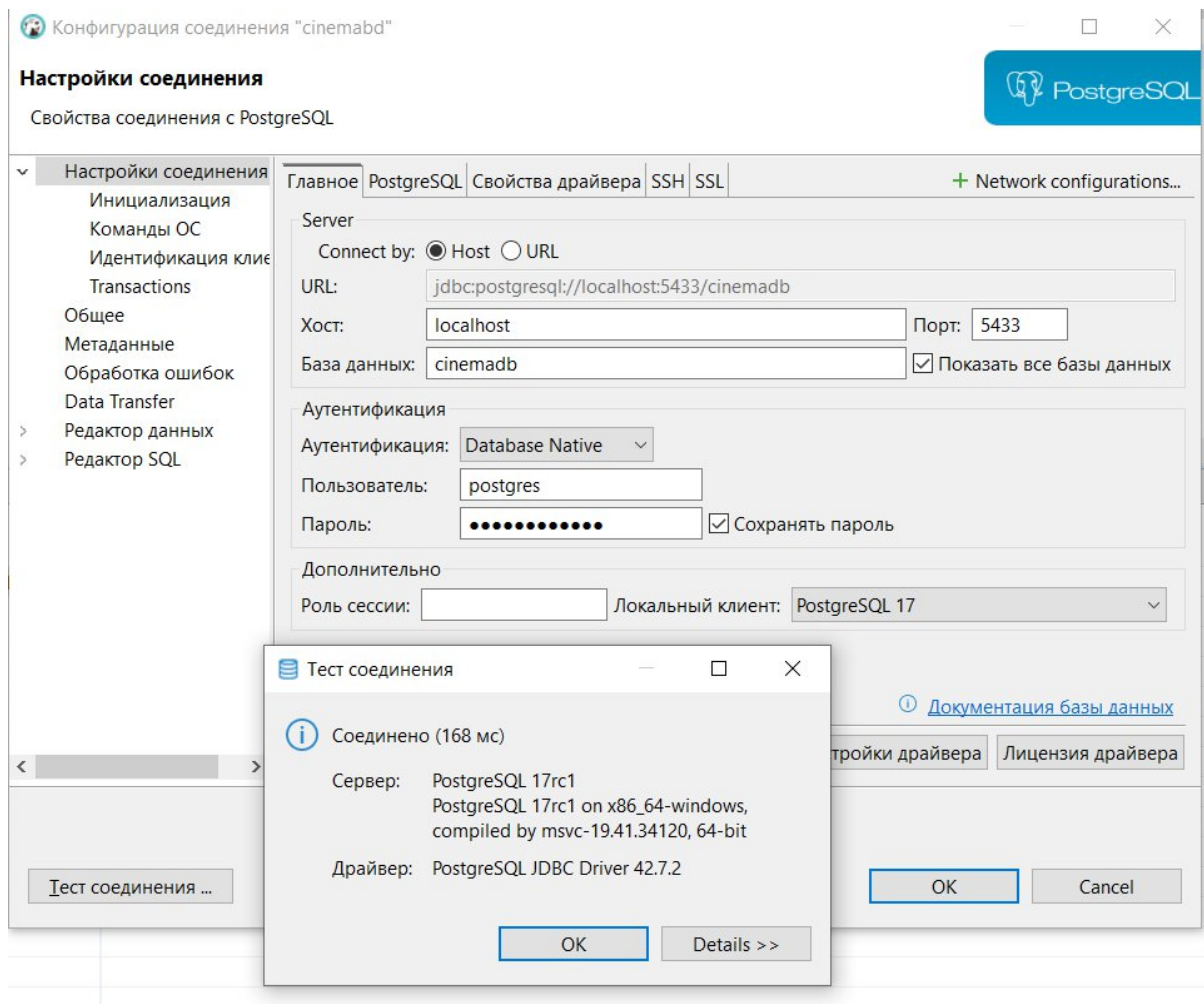
После выполнения этой команды можно обновить список баз данных в DBeaver, чтобы увидеть созданную базу данных.



Подключимся к ней, чтобы создать сами таблицы. Для этого нужно перейти в раздел «Базы данных» и выбрать пункт «Новое соединение»:



Откроется меню, где нужно ввести данные. Самое важное — указать название базы данных:



3. Создаём таблицу Movies. Для начала напишем команду CREATE TABLE, которая создаст таблицу Movies с колонками для идентификатора фильма, названия, года выпуска, жанра и рейтинга:

```
CREATE TABLE Movies (  
  movie_id INT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  release_year INT,  
  genre VARCHAR(255),  
  rating DECIMAL(3, 1)  
);
```

В этой таблице определяем столбец movie_id как уникальный идентификатор фильма с типом данных INT и устанавливаем его в качестве первичного ключа. Это значит, что каждое значение в этом столбце будет уникальным для каждой записи, что позволит однозначно идентифицировать каждый фильм.

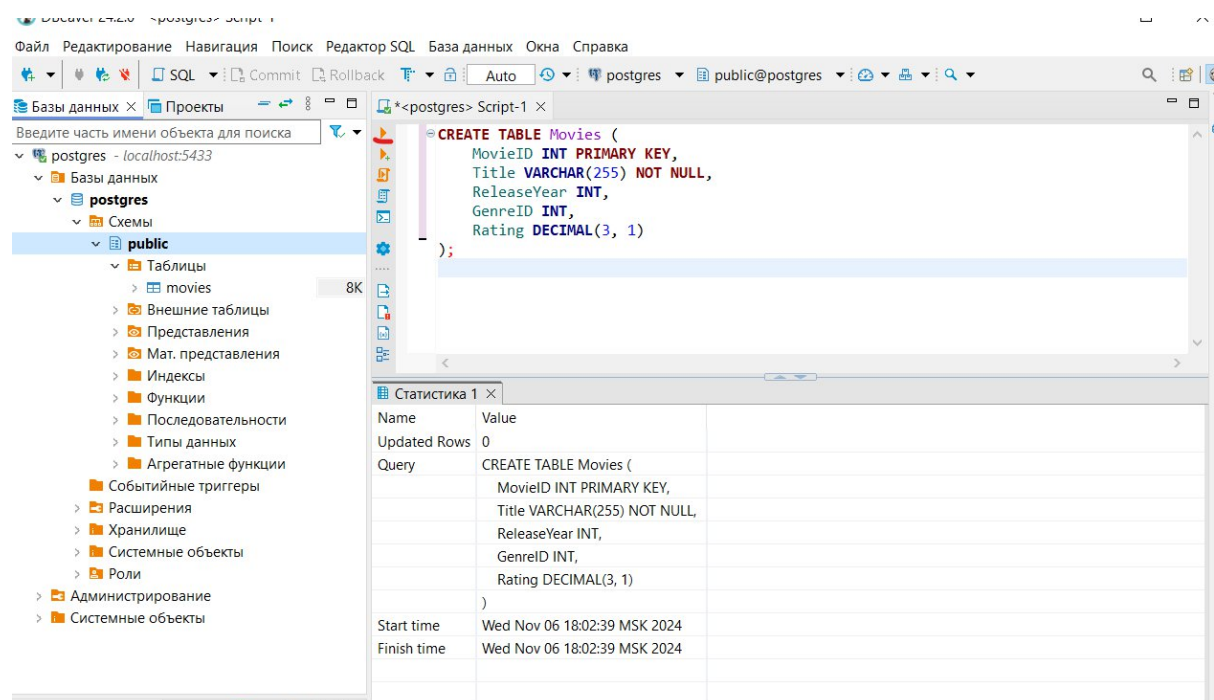
Столбец `title` предназначен для хранения названия фильма. Он имеет тип данных `VARCHAR(255)`, что позволяет сохранять строки длиной до 255 символов. Это поле также имеет ограничение `NOT NULL`, что означает, что название фильма обязательно должно быть указано.

Год выпуска фильма будет храниться в столбце `release_year` с типом данных `INT`, что подходит для целых чисел. Это поможет нам отслеживать даты выхода фильмов и облегчит фильтрацию по годам.

Также мы добавляем столбец `genre`, который будет содержать жанр фильма. Его тип данных — `VARCHAR(255)`, что позволяет хранить текстовые значения, описывающие жанр. В будущем, если потребуется, этот столбец можно будет заменить на идентификатор для связи с отдельной таблицей жанров, чтобы определить, к какому жанру принадлежит каждый фильм.

Наконец, в таблице создаётся столбец `rating`, который будет хранить рейтинг фильма. Его тип данных — `DECIMAL(3, 1)`, что позволяет хранить числа с одной цифрой после запятой. Этот формат пригодится для представления рейтингов, которые могут варьироваться, например, от 0.0 до 10.0. Таким образом, таблица `Movies` будет содержать всю необходимую информацию о фильмах, что облегчит дальнейшую работу с данными.

4. Выполняем запрос и проверяем результат. Нажимаем «Выполнить», чтобы создать таблицу. После этого можно обновить список таблиц в DBViewer и убедиться, что таблица `Movies` создана.



Теперь создадим таблицу **Actors**, чтобы хранить данные об актёрах:

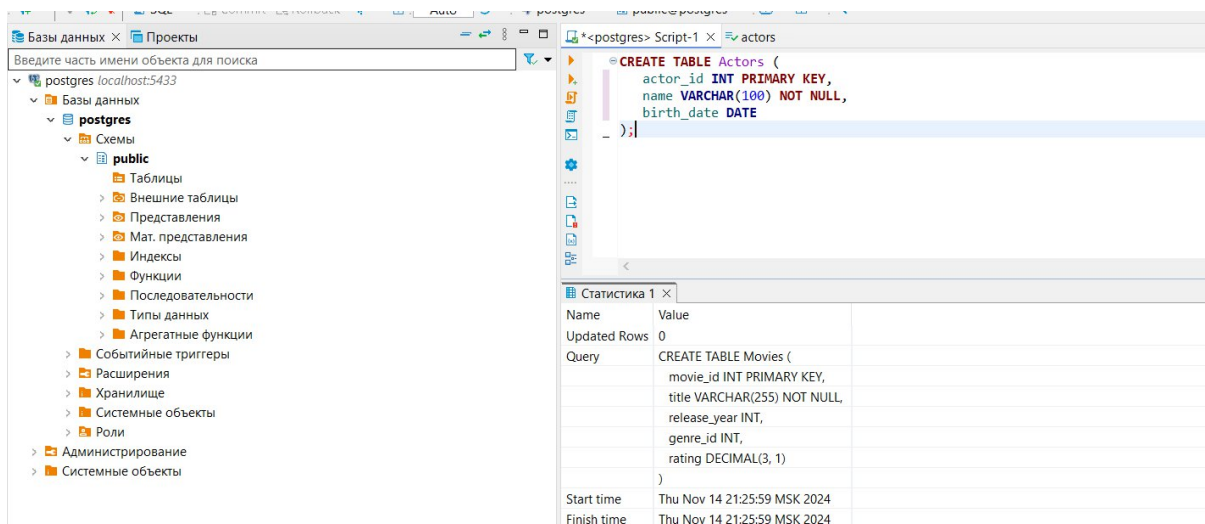
```
CREATE TABLE Actors (
  actor_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  birth_date DATE
);
```

В этой таблице:

- ActorID — уникальный идентификатор актёра.
- Name — имя актёра, также с ограничением NOT NULL.
- BirthDate — дата рождения актёра, тип данных — DATE.

Теперь проверим успешное создание таблиц и добавим несколько записей для тестирования, используя графический интерфейс, как мы делали в предыдущих уроках. Следуйте этим шагам:

1. В левом меню найдите раздел с таблицами и нажмите Refresh («Обновить»), если таблицы Actors и Movies не отобразились после создания.



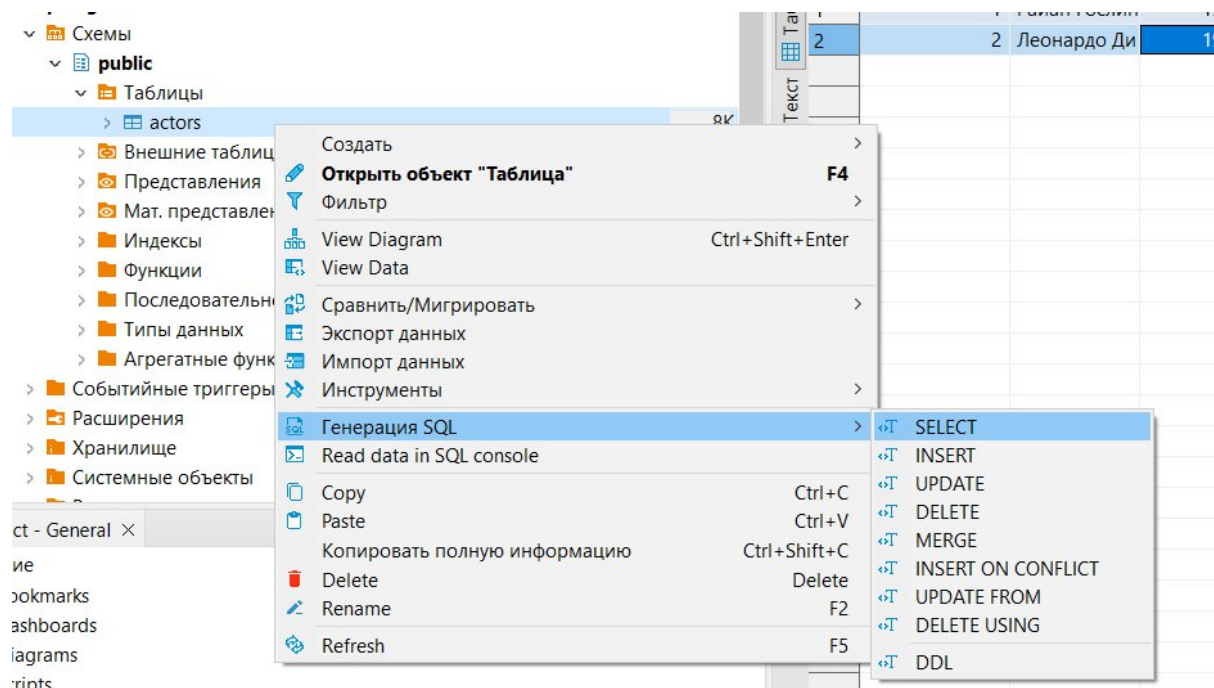
The screenshot shows the pgAdmin interface. On the left, the 'Базы данных' (Databases) pane is expanded, showing the 'postgres' database. Under 'Схемы' (Schemas), the 'public' schema is selected, and the 'Таблицы' (Tables) folder is expanded. The main pane shows the 'CREATE TABLE Actors' query. Below the query, the 'Статистика 1' (Statistics 1) window is open, displaying the following information:

Name	Value
Updated Rows	0
Query	CREATE TABLE Movies (movie_id INT PRIMARY KEY, title VARCHAR(255) NOT NULL, release_year INT, genre_id INT, rating DECIMAL(3, 1))
Start time	Thu Nov 14 21:25:59 MSK 2024
Finish time	Thu Nov 14 21:25:59 MSK 2024

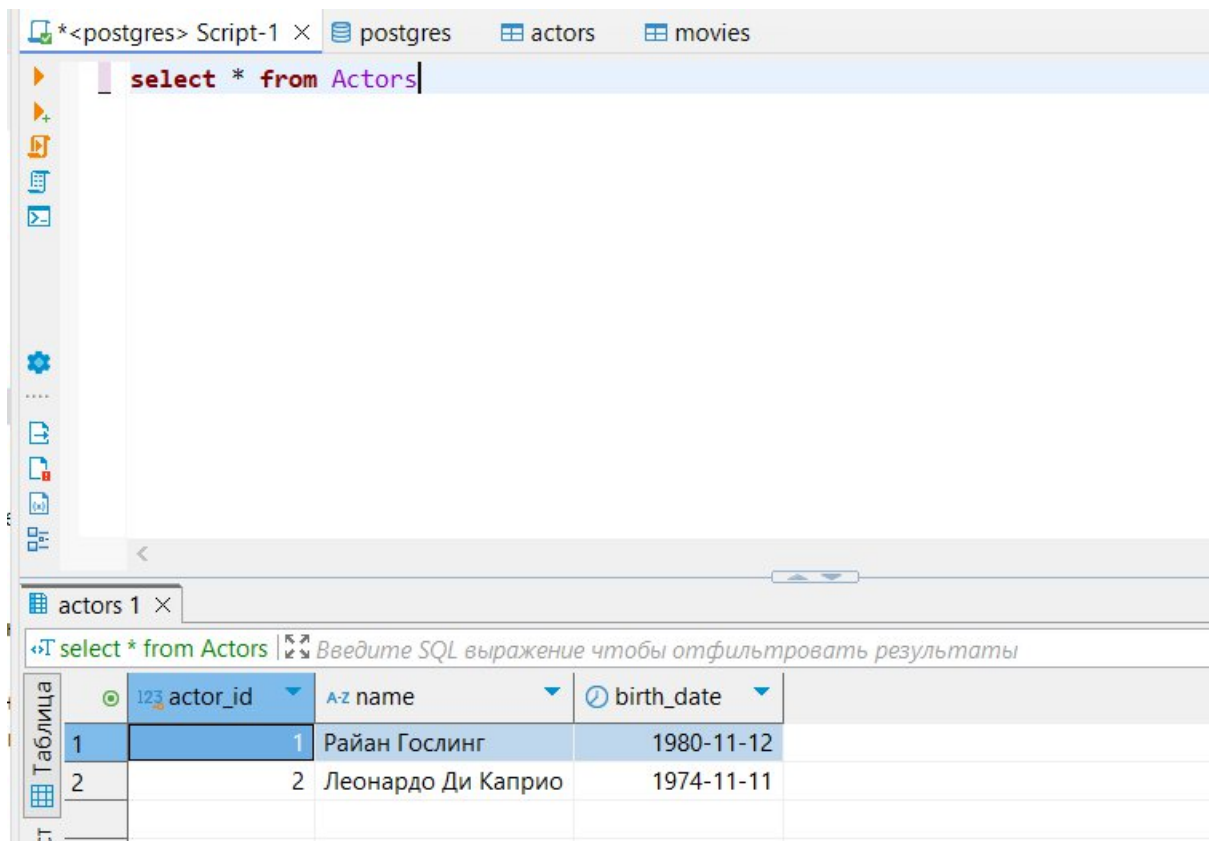
- Аналогичным образом откройте таблицу Movies, добавьте несколько фильмов, заполнив соответствующие поля movie_id, title, release_year, genre_id и rating:
 - 1, Матрица, 1999, научно-фантастический боевик, 8.5
 - 2, Однажды в... Голливуде, 2019, Комедия, 7.7

	movie_id	title	release_year	genre	rating
1	1	Матрица	1999	научно-фантастический боевик	8,5
2	2	Однажды в... Голливуде	2019	Комедия	7,7

4. Сгенерируем SELECT-запрос для вывода данных. Для этого нажмите на правую кнопку мыши → «Генерация SQL» → SELECT:



Скопируем код и перенесём его в SQL-скрипт:



Эти команды отобразят все данные, добавленные в таблицы Actors и Movies, и помогут убедиться, что записи были успешно внесены.

Команда ALTER

Но что делать, если нужно изменить значения в таблицах? Для этого используется команда [ALTER TABLE](#). Она позволяет добавлять, изменять и удалять колонки, а также применять ограничения.

Основные операции с ALTER TABLE:

1. Добавление нового столбца.

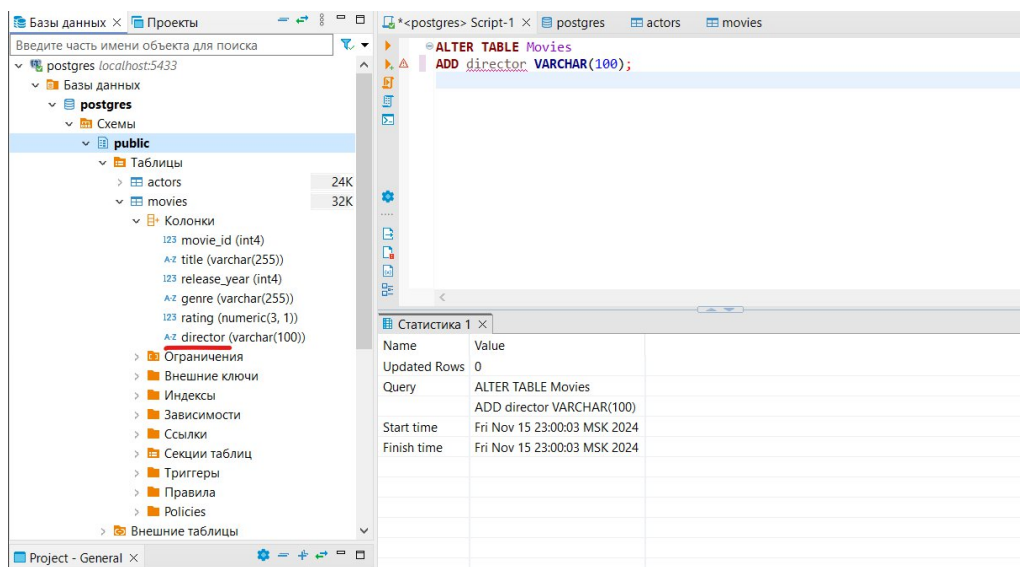
Синтаксис команды:

```
ALTER TABLE table_name  
ADD column_name datatype constraint;
```

- ALTER TABLE — модификация колонки.
- table_name — наименование таблицы.
- ADD — добавление нового столбца в таблицу.
- column_name — название нового столбца.
- datatype — тип данных в столбце для нового столбца.
- constraint — ограничения для нового столбца (необязательно).

Например, если нам нужно добавить новый столбец для хранения имени режиссёра в таблицу Movies, можно сделать это с помощью команды ALTER TABLE для добавления столбца director. Это позволит сохранить информацию о режиссёре каждого фильма.

```
ALTER TABLE Movies  
ADD director VARCHAR(100);
```



2. Изменение типа данных столбца.

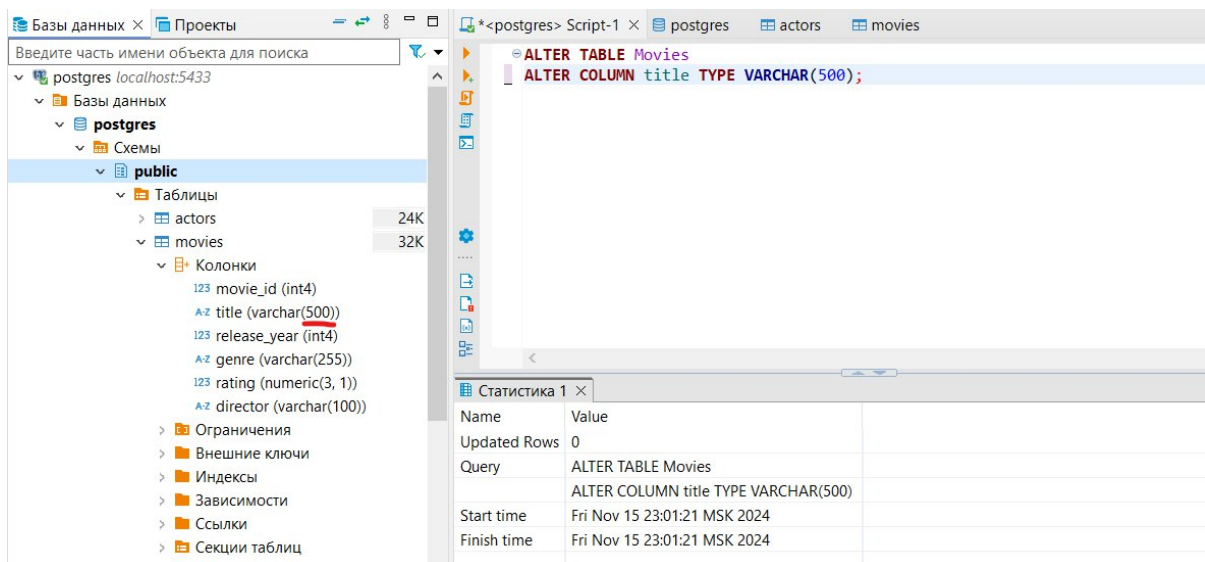
Синтаксис команды:

```
ALTER TABLE table_name  
ALTER COLUMN column_name new_datatype;
```

- ALTER TABLE — команда для изменения существующей таблицы.
- table_name — наименование таблицы, в которой будет изменён столбец.
- ALTER COLUMN — изменение типа данных или других характеристик столбца.
- column_name — имя столбца, который нужно изменить.
- new_datatype — новый тип данных, который будет присвоен столбцу (например, VARCHAR(100), INT, DATE).

Если в будущем потребуется изменить существующий столбец, например увеличить максимальную длину названия фильма в столбце title, мы также можем воспользоваться командой ALTER TABLE, изменив тип данных этого столбца. Это особенно полезно, если названия фильмов со временем становятся длиннее, а текущего ограничения недостаточно.

```
ALTER TABLE Movies  
ALTER COLUMN title TYPE VARCHAR(500);
```



3. Удаление столбца.

Синтаксис команды:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

- ALTER TABLE — модификация колонки.
- table_name — наименование таблицы.
- DROP COLUMN — удаление указанного столбца из таблицы.
- column_name — название столбца, который удаляем.

Иногда может потребоваться удалить ненужный столбец, чтобы упростить таблицу или освободить место. Например, если больше не нужно хранить информацию о режиссёре, можно удалить столбец director.

```
ALTER TABLE Movies  
DROP COLUMN director;
```

The screenshot shows a PostgreSQL client interface. On the left, a tree view displays the database structure: postgres (localhost:5433) > Базы данных > postgres > Схемы > public > Таблицы. The 'movies' table is selected, showing its columns: movie_id (int4), title (varchar(500)), release_year (int4), genre (varchar(255)), and rating (numeric(3, 1)).

The main window shows a query editor with the following SQL code:

```
ALTER TABLE Movies  
DROP COLUMN director;
```

Below the query editor, a 'Статистика 1' (Statistics 1) window is open, displaying the following data:

Name	Value
Updated Rows	0
Query	ALTER TABLE Movies DROP COLUMN director
Start time	Fri Nov 15 23:01:51 MSK 2024
Finish time	Fri Nov 15 23:01:51 MSK 2024

Связь между таблицами через внешние ключи

В нашей базе данных две таблицы: Movies и Actors. Одна из них хранит информацию о фильмах, другая — об актёрах. Однако в реальной жизни эти данные связаны: в фильмах всегда снимаются актёры. Логично, что и в нашей базе данных они должны быть связаны между собой, чтобы данные отражали эту взаимосвязь.

Например, мы хотим знать, какой актёр исполняет главную роль в каждом фильме. Чтобы это было удобно и структурировано, используется механизм внешних связей (Foreign Keys). Связи между таблицами позволяют создавать логическую связь, чтобы информация была неразрывно связана, а ошибки ввода данных минимизированы.

Подробно разберём, как можно связать таблицы Movies и Actors с помощью внешнего ключа.

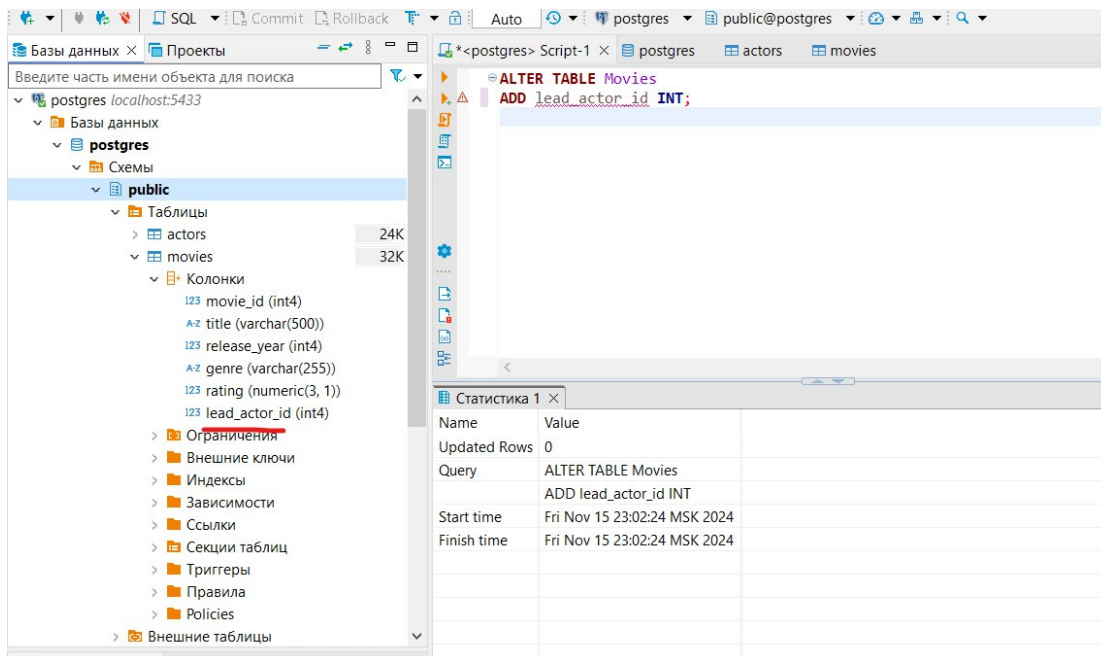
1. Добавляем столбец для связи.

Чтобы указать, какой актёр играет ведущую роль в фильме, в таблицу Movies нужно добавить новый столбец — `lead_actor_id`. Этот столбец будет содержать идентификатор актёра из таблицы Actors.

```
ALTER TABLE Movies  
ADD lead_actor_id INT;
```

Что происходит на этом этапе?

- В таблице Movies появляется дополнительный столбец.
- Столбец будет хранить числовые значения, соответствующие идентификаторам актёров (`actor_id`) из таблицы Actors.



2. Устанавливаем внешний ключ.

После добавления столбца мы должны указать базе данных, что значения в `lead_actor_id` должны быть связаны с уже существующими значениями в таблице `Actors`. Это делается с помощью создания внешнего ключа.

```
ALTER TABLE Movies
ADD FOREIGN KEY (lead_actor_id) REFERENCES Actors(actor_id);
```

Что делает эта команда?

- Она связывает `lead_actor_id` в таблице `Movies` со столбцом `actor_id` в таблице `Actors`.
- Теперь база данных проверяет, что любое значение, добавленное в `lead_actor_id`, обязательно существует в `actor_id`.

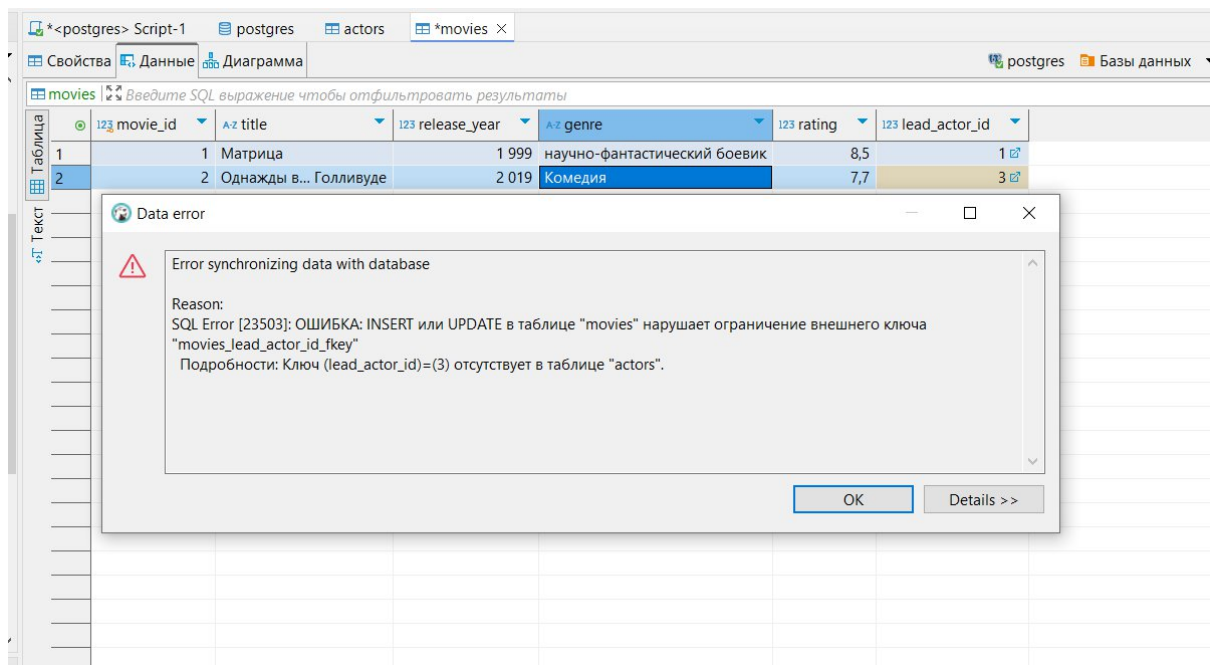
Почему это важно?

1. Целостность данных. Внешний ключ гарантирует, что данные в таблице `Movies` будут корректно связаны с таблицей `Actors`. Если мы попытаемся добавить фильм с актёром, которого нет в таблице `Actors`, база данных выдаст ошибку. Это предотвращает появление «мусорных» данных.

2. Экономия места и снижение дублирования данных. Вместо того чтобы хранить имя или другие подробности об актёре прямо в таблице `Movies`, достаточно сохранить его идентификатор (`actor_id`). Это упрощает обновление данных: если имя актёра изменится, достаточно обновить его в таблице `Actors`, и изменения автоматически отразятся везде, где упоминается этот актёр.

3. Логическая организация. Таблицы остаются независимыми, но логически связанными. Мы можем легко извлечь данные (например, узнать, какой актёр играет главную роль в конкретном фильме), используя запрос с объединением таблиц.

После того как мы добавили внешний ключ, можно проверить, как работает связь. Например, если мы попытаемся добавить в Movies значение `lead_actor_id`, которого нет в Actors, база данных вернёт ошибку:



Это поведение подтверждает, что связь настроена правильно и данные сохраняются в целостности.

А теперь исправим ошибку и поставим существующие значения:

